

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи № 2
з курсу “Теорія обчислень, алгоритми та структури даних”
«Швидке сортування. Порядкові статистики»

Виконав:
студент групи ФеІ-11
Баран М.Т.
Перевірив:
ас. Баран.М.О.

Львів-2026

Мета роботи

Ознайомитись з методами сортування масивів та реалізувати метод швидкого сортування.

Теоретичні відомості

1. Загальне поняття та історія

Швидке сортування (Quick Sort) — це один із найвідоміших, найшвидших та найчастіше використовуваних на практиці алгоритмів сортування масивів. Його розробив британський інформатик Чарльз Ентоні Річард Хоар (Tony Hoare) у 1959 році (опублікував у 1961 році).

Алгоритм базується на фундаментальній парадигмі програмування **«розділяй і володарюй» (divide and conquer)**. Суть цієї парадигми полягає в розбитті складної задачі на кілька менших підзадач того ж типу, їх розв'язанні та подальшому об'єднанні результатів.

2. Принцип роботи алгоритму

Процес швидкого сортування можна звести до трьох основних кроків:

1. **Вибір опорного елемента (Pivot).** З масиву обирається один елемент, який називається опорним (або «півотом»). Існують різні стратегії вибору цього елемента (наприклад, перший, останній, центральний або випадковий).
2. **Розбиття (Partitioning).** Масив перерозподіляється таким чином, що всі елементи, які менші за опорний, переміщуються ліворуч від нього, а всі елементи, які більші (або рівні) — праворуч. Після цього кроку опорний елемент опиняється на своїй остаточній, відсортованій позиції.
3. **Рекурсія.** Алгоритм рекурсивно застосовує кроки 1 і 2 до лівого та правого підмасивів, утворених після розбиття. Базовим випадком рекурсії є масив із одного або нуля елементів — такий масив уже вважається відсортованим.

3. Оцінка складності (Час та Пам'ять)

Ефективність алгоритму сильно залежить від того, наскільки вдало обирається опорний елемент на кожному кроці розбиття.

- **Найкращий випадок:** $O(n \log n)$. Досягається тоді, коли опорний елемент щоразу ділить масив рівно навпіл (або на дві майже рівні частини). Глибина рекурсії становить $\log_2 n$, і на кожному рівні виконується близько n операцій порівняння.
- **Середній випадок:** $O(n \log n)$. Навіть при не ідеально рівному розбитті (наприклад, у пропорції 9:1), алгоритм все одно працює з логарифмічною глибиною рекурсії. Саме тому на практиці Quick Sort перевершує більшість інших алгоритмів із такою ж асимптотикою (наприклад, Merge Sort або Heap Sort) завдяки меншій константі та дружньому ставленню до кешу процесора.
- **Найгірший випадок:** $O(n^2)$. Виникає, коли на кожному етапі масив ділиться на одну частину довжиною $n-1$, а іншу — довжиною 0. Це типово відбувається, якщо масив уже відсортований (або відсортований у зворотному порядку), а в якості опорного постійно обирається перший або останній елемент.
- **Просторова складність:** $O(\log n)$ у найкращому та середньому випадках, і $O(n)$ у найгіршому. Хоча алгоритм сортує дані на місці (in-place) і не потребує створення копій масиву, додаткова пам'ять витрачається на підтримку стека викликів рекурсії.

4. Стратегії вибору опорного елемента (Pivot)

Щоб уникнути найгіршого випадку $O(n^2)$, розроблено кілька підходів до вибору півота:

- **Завжди перший або останній елемент:** Найпростіший в реалізації, але найвразливіший до частково відсортованих даних.
- **Випадковий елемент (Randomized Quick Sort):** Опорний елемент обирається випадково. Це робить виникнення найгіршого випадку вкрай мало ймовірним.
- **Медіана трьох (Median-of-three):** В якості опорного обирається медіана серед першого, середнього та останнього елементів масиву. Це ефективний компроміс, який значно покращує роботу алгоритму на практиці без зайвих обчислювальних витрат на генерацію випадкових чисел.

5. Ключові властивості алгоритму

- **На місці (In-place):** Алгоритм не потребує виділення додаткової пам'яті розміром $O(n)$ для створення нових масивів (на відміну від сортування злиттям - Merge Sort). Всі перестановки відбуваються всередині вихідного масиву.
- **Нестійкість (Unstable):** Quick Sort не є стійким сортуванням. Це означає, що елементи з однаковими ключами (значеннями) можуть змінити своє відносне розташування один щодо одного після завершення сортування.
- **Кеш-ефективність:** Оскільки алгоритм послідовно сканує масив під час процедури розбиття, він дуже ефективно використовує кеш-пам'ять процесора (має високий рівень локальності даних).

Хід роботи

1. Створити бібліотеку *Sort* (файли *Sort.h*, *Sort.cpp*).
2. У бібліотеці *Sort* (файл *Sort.cpp*) реалізуємо функції QuickSort та Partition, RandomizedPartition, RandomizedSelect

```
void QuickSort(int* A, int first, int last)
{
    if (first < last)
    {
        int pivot;
        Partition(A, first, last, pivot);
        QuickSort(A, first, pivot - 1);
        QuickSort(A, pivot + 1, last);
    }
}

void Partition(int* A, int first, int last, int& pivot)
{
    int x = A[last];
    int i = first - 1;
    for (int j = first; j < last; j++)
    {
        if (A[j] ≤ x)
        {
            i++;
            my_swap(A[i], A[j]);
        }
    }
    my_swap(A[i + 1], A[last]);
    pivot = i + 1;
}
```

```

void RandomizedPartition(int* A, int first, int last, int& pivot)
{
    int random = first + rand() % (last - first + 1);
    my_swap(A[random], A[last]);
    Partition(A, first, last, pivot);
}

int RandomizedSelect(int A[], int p, int r, int i)
{
    if (p == r)
        return A[p];
    int q;
    RandomizedPartition(A, p, r, q);
    int k = q - p + 1;
    if (i == k)
        return A[q];
    else if (i < k)
        return RandomizedSelect(A, p, q - 1, i);
    else
        return RandomizedSelect(A, q + 1, r, i - k);
}

```

3. У заголовному файлі *Sort.h* запрограмуємо інтерфейси цих функцій.

```

//Methods Lab_2
void QuickSort(int* A, int first, int last);
void Partition(int* A, int first, int last, int& pivot);
void RandomizedPartition(int* A, int first, int last, int& pivot);
int RandomizedSelect(int A[], int p, int r, int i);

```

4. Створюємо новий проект *Lab_01* та підключаємо до нього бібліотеку *Sort.h*
5. У функції *main()* проекту реалізуємо можливість введення одновимірного масиву даних та відображення результатів сортування

```

int n = 0;

std::cout << "Введіть кількість елементів :";
std::cin >> n;

if (n ≤ 0) {
    std::cout << "Некоректне число" << std::endl;
    return 1;
}

int* A = new int[n];
int* B = new int[n];

std::cout << "Введіть елементи :" << std::endl;
for (int i = 0; i < n; i++) {
    if (!(std::cin >> A[i])) {
        std::cout << "Помилка: введіть число!" << std::endl;
        std::cin.clear();
        std::cin.ignore(10000, '\n');
        --i;
    }
}

std::cout << "Масив перед сортуванням:" << std::endl;
show(A, n);

std::cout << "Масив після сортування:" << std::endl;
QuickSort(A, 0, n - 1);
show(A, n);

```

```

int i_stat;
std::cout << "\nВведіть номер порядкової статистики (1... " << n << "): ";
std::cin >> i_stat;
int result = RandomizedSelect(A, 0, n - 1, i_stat);
std::cout << i_stat << "-та порядкова статистика: " << result << "\n";

int min = RandomizedSelect(A, 0, n - 1, 1);
std::cout << "Мінімальне значення: " << min << "\n";

int max = RandomizedSelect(A, 0, n - 1, n);
std::cout << "Максимальне значення: " << max << "\n";

if (n % 2 == 1)
{
    int median = RandomizedSelect(A, 0, n - 1, (n + 1) / 2);
    std::cout << "Медіана: " << median << "\n";
}
else
{
    int m1 = RandomizedSelect(A, 0, n - 1, n / 2);
    int m2 = RandomizedSelect(A, 0, n - 1, n / 2 + 1);
    std::cout << "Медіани: " << m1 << " і " << m2 << "\n";
}

```

6. Компілюємо проект та запускаємо програму на виконання

```
Введіть кількість елементів : 5
Введіть елементи :
1
4
2
3
8
Масив перед сортуванням:
1 4 2 3 8
Масив після сортування:
1 2 3 4 8

Введіть номер порядкової статистики (1...5): 3
3-та порядкова статистика: 3
Мінімальне значення: 1
Максимальне значення: 8
Медіана: 3
```

Висновок

У ході лабораторної роботи реалізовано та досліджено алгоритм Quick Sort. Основні результати:

1. **Ефективність:** Підхід «розділяй і володарюй» із розбиттям масиву навколо опорного елемента (pivot) значно скорочує кількість порівнянь.
2. **Складність:** Середній час виконання становить $O(n \log n)$. У найгіршому випадку (за невдалого вибору півота) складність падає до $O(n^2)$.
3. **Вибір півота:** Використання оптимізованих стратегій (випадковий елемент або «медіана трьох») критично важливе для уникнення найгіршого сценарію.
4. **Пам'ять:** Сортування виконується «на місці» (in-place) без дублювання масиву, вимагаючи в середньому лише $O(\log n)$ додаткової пам'яті для стека рекурсії.
5. **Застосування:** Попри нестійкість алгоритму, його висока швидкодія та кеш-ефективність роблять його оптимальним вибором для більшості задач сортування.

Код програми

Sort.h

```
void QuickSort(int* A, int first, int last);
void Partition(int* A, int first, int last, int& pivot);
void RandomizedPartition(int* A, int first, int last, int& pivot);
int RandomizedSelect(int A[], int p, int r, int i);
```

Sort.cpp

```
void QuickSort(int* A, int first, int last)
{
    if (first < last)
    {
        int pivot;
        Partition(A, first, last, pivot);
        QuickSort(A, first, pivot - 1);
        QuickSort(A, pivot + 1, last);
    }
}

void Partition(int* A, int first, int last, int& pivot)
{
    int x = A[last];
    int i = first - 1;
    for (int j = first; j < last; j++)
    {
        if (A[j] <= x)
        {
            i++;
            my_swap(A[i], A[j]);
        }
    }
    my_swap(A[i + 1], A[last]);
    pivot = i + 1;
}

void RandomizedPartition(int* A, int first, int last, int& pivot)
{
    int random = first + rand() % (last - first + 1);
    my_swap(A[random], A[last]);
    Partition(A, first, last, pivot);
}

int RandomizedSelect(int A[], int p, int r, int i)
{
    if (p == r)
        return A[p];
    int q;
    RandomizedPartition(A, p, r, q);
    int k = q - p + 1;
    if (i == k)
        return A[q];
    else if (i < k)
        return RandomizedSelect(A, p, q - 1, i);
    else
        return RandomizedSelect(A, q + 1, r, i - k);
}
```

Lab_02.cpp

```
#include <iostream>
#include "Sort.h"
#include <windows.h>
#include <ctime>

int main()
{
    srand(static_cast<unsigned int>(time(0)));
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    int n = 0;

    std::cout << "Введіть кількість елементів :";
    std::cin >> n;

    if (n <= 0) {
        std::cout << "Некоректне число" << std::endl;
        return 1;
    }

    int* A = new int[n];
    int* B = new int[n];

    std::cout << "Введіть елементи :" << std::endl;
    for (int i = 0; i < n; i++) {
        if (!(std::cin >> A[i])) {
            std::cout << "Помилка: введіть число!" << std::endl;
            std::cin.clear();
            std::cin.ignore(10000, '\n');
            --i;
        }
    }

    std::cout << "Масив перед сортуванням:" << std::endl;
    show(A, n);

    std::cout << "Масив після сортування:" << std::endl;
    QuickSort(A, 0, n - 1);
    show(A, n);

    int i_stat;
    std::cout << "\nВведіть номер порядкової статистики (1..." << n << "):";
    std::cin >> i_stat;
    int result = RandomizedSelect(A, 0, n - 1, i_stat);
    std::cout << i_stat << "-та порядкова статистика: " << result << "\n";

    int min = RandomizedSelect(A, 0, n - 1, 1);
    std::cout << "Мінімальне значення: " << min << "\n";
```



```

int max = RandomizedSelect(A, 0, n - 1, n);
std::cout << "Максимальне значення: " << max << "\n";

if (n % 2 == 1)
{
    int median = RandomizedSelect(A, 0, n - 1, (n + 1) / 2);
    std::cout << "Медіана: " << median << "\n";
}
else
{
    int m1 = RandomizedSelect(A, 0, n - 1, n / 2);
    int m2 = RandomizedSelect(A, 0, n - 1, n / 2 + 1);
    std::cout << "Медіани: " << m1 << " i " << m2 << "\n";
}

delete[] A;
delete[] B;
}

```